

Department of Computer Science and Engineering
Indian Institute of Technology Jodhpur
CSL7490: Introduction to Blockchain (Major Exam)

Time: 3:00 hrs. Date: 25 April 2026, Saturday Maximum Marks: 80

Roll Number _____

Name: _____

Please read the following instructions carefully.

- All questions are compulsory.
- Clearly and concretely list all assumptions if you are making any.
- A direct answer (result) without any accompanying explanation will receive 0 marks.
- Please aim for precision in your answer to the question. Any unnecessary elaboration or attempt to deceive will result in a 50% deduction in marks.
- If anyone is found copying before, during, or after the exam, he/she will be removed from the course and given a FAILED grade.

Part - A

Note: Answers for Part-A must be written directly on the question paper.

1. Select the correct option for each of the following multiple-choice questions.

- (a) A Merkle Tree Second Preimage Attack is most effective when the system fails to implement which security measure? [1 Mark]
- i. Domain Separation
 - ii. Using SHA-3 instead of Keccak-256.
 - iii. Salted hashing for every transaction.
 - iv. Increasing the depth of the tree beyond 32 levels.

Answer: i. Domain Separation (e.g., prepending 0x00 to leaves and 0x01 to internal nodes). Without domain separation, a node might interpret a concatenated pair of hashes as a single leaf's raw data.

- (b) Bitcoin Script is described as Forth-like and Turing-incomplete. What is the primary security benefit of it being Turing-incomplete? [1 Mark]
- i. It ensures that only the owner of the private key can spend the funds.
 - ii. It makes the scripts smaller and easier to store in the UTXO set.
 - iii. It preventing infinite loop DoS attacks.
 - iv. It prevents users from writing complex smart contracts.

Answer: iii.

- (c) In Bitcoin's P2SH (Pay-to-Script-Hash), when does the network first see the actual Redeem Script logic? [1 Mark]
- i. When the UTXO is first created (funding transaction).
 - ii. When the miner includes the transaction in the Merkle Tree.
 - iii. During the difficulty adjustment period.
 - iv. Only when the funds are being spent.

Answer: iv. This is a key privacy and efficiency feature. The sender only sends money to a hash. The actual logic (e.g., a 2-of-3 multisig) is only revealed by the spender when they provide the ScriptSig to unlock the funds.

(d) What is the MSP (Membership Service Provider) in Hyperledger Fabric? [1 Mark]

- i. A specialized node that orders transactions.
- ii. It defines the rules for verifying identities and assigning roles.
- iii. A database that stores the private keys of all users in the network.
- iv. A consensus plugin that replaces PBFT in small networks.

Answer: ii. The MSP is how Fabric handles Permissioning. It defines which Certificate Authorities are trusted and which certificates correspond to Admins, Peers, or Clients.

(e) Why does Gasper separate the Justification and Finalization steps instead of moving directly to finality? [1 Mark]

- i. To ensure that the RANDAO reveal has enough entropy.
- ii. Because BLS signatures can only be aggregated once per epoch.
- iii. To allow time for the Execution Layer to verify transaction nonces.
- iv. To prevent forks due to network delays in an asynchronous network.

Answer: iv.

2. Fill in the blanks in the following questions.

(a) The _____ field in the Bitcoin block header was originally 32 bits, but due to rising hash rates, miners now also use the extraNonce located in the _____ transaction to find valid blocks. [1 Mark]

Answer: Nonce, Coinbase

(b) In the Merkle Patricia Tree, a _____ Node is used when a specific path in the trie terminates at a unique key-value pair, containing the remaining suffix of the key. [1 Mark]

Answer: Leaf

(c) In a PBFT system, if f is the number of Byzantine nodes, the system requires a total of $3f + 1$ nodes to ensure that the non-faulty nodes (which number $2f + 1$) can always outvote the faulty ones and reach a _____ of $2f + 1$. [1 Mark]

Answer: Quorum

(d) The EVM uses a _____ bit word size and has a maximum stack depth of _____ elements. If a function call attempts to push the _____ element onto the stack, a _____ error occurs. [1 Mark]

Answer: 256-bit, 1024, 1025th (or > 1025), stack overflow

(e) Hyperledger Fabric uses a _____ model, where transaction ordering, validation, and execution are separated. [1 Mark]

Answer: execute-order-validate

3. Write short answers for the following questions.

(a) A developer wants a transaction that requires a specific user's signature while keeping the spending conditions hidden until the output is spent. Which Bitcoin script type is best suited for this purpose? [1 Mark]

Answer: P2SH (Pay to Script Hash). It hides the full redeem script until the output is spent, revealing the logic only at spending time.

(b) What is the primary difference between Pay to Public Key (P2PK) and Pay to Public Key Hash (P2PKH) in terms of what is revealed on the public ledger before a transaction is spent? [1 Mark]

Answer: P2PK reveals the public key immediately; P2PKH only reveals the hash of the public key until the funds are spent.

(c) In a distributed system where nodes may unexpectedly stop participating but are assumed not to act maliciously or send incorrect data, a protocol is required to ensure agreement on a single value. Given that adversarial behavior is not part of the threat model, which class of consensus protocols is most appropriate for such a setting? [1 Mark]

Answer: Paxos (or Raft) designed for Crash Fault Tolerant (CFT) environments.

- (d) In PBFT (Practical Byzantine Fault Tolerance), why are three phases (Pre-prepare, Prepare, Commit) necessary to reach consensus? [1 Mark]
 Answer: To ensure that all non-faulty nodes agree on a total **ordering of requests** even if the primary node is malicious.
- (e) In the Gasper protocol, what happens during an Inactivity Leak? [1 Mark]
 Answer: Offline validators gradually lose their stake until the online validators regain a 2/3 majority to finalize blocks.
- (f) What is the View Change protocol in PBFT designed to solve? [1 Mark]
 Answer: It allows the network to elect a new Primary node if the current one is suspected of being faulty or malicious (liveness protection).
- (g) In Paxos, what is the primary purpose of the Promise sent by an Acceptor to a Proposer? [1 Mark]
 Answer: It is a commitment not to accept any future proposals with a lower sequence number than the current one. This prevents old proposals from overwriting new consensus decisions.
- (h) An auditor is auditing a company database that uses an ECDSA-based system and notices that the randomness used during signing is identical across multiple signatures for different messages. What is the immediate consequence of this, if the auditor is malicious? [1 Mark]
 Answer: An auditor can perform simple algebra to solve for the private key using two different signatures. If the randomness or nonce is reused, $d = (s_1k - z_1) \cdot r^{-1} \pmod{n}$.
- (i) A contract is deployed to the network. During the deployment transaction, the constructor logic is executed. What is actually stored on-chain in the contract account's code field? [1 Mark]
 Answer: The constructor logic is part of the init code and is discarded after it returns the runtime code.
- (j) If a contract's constructor runs a very expensive loop and runs out of gas, what happens to the address that was pre-computed for that contract? [1 Mark]
 Answer: The deployment fails, the transaction reverts, no code is stored, and the nonce for the creator address is still incremented.

Part - B

1. To compute the public key $Q = kG$, a hardware wallet uses the Binary Add-and-Double method. For a private key $k = 233$, identify the number of **Double** and **Addition** operations required. If a Double takes $12\mu s$ and an Addition takes $18\mu s$, calculate the total time. [3 Marks]

Answer:

- **Binary Representation of 233:**

To find the binary of 233:

$$233 = 128 + 64 + 32 + 8 + 1$$

$$233 = 2^7 + 2^6 + 2^5 + 2^3 + 2^0$$

Binary:

$$(11101001)_2$$

- **Trace of Algorithm (Table 1):**

- **Final Operation Count:**

Doublings (D): 7 (Every bit after the first bit triggers a Double).

Additions (A): 4 (Every bit after the first bit that is a '1' triggers an Add).

Total Operations: 11

- **Total Execution Time Calculation:** Given the specific hardware constraints for the scalar $k = 233$:

– Time for one Double (T_D) = $12\mu s$

– Time for one Addition (T_A) = $18\mu s$

Table 1: Trace of the Left-to-Right Double-and-Add Algorithm for $k = 233$

Bit Index	Bit Value	Operation	Current Result (Q)
Bit 7	1	Initialize	P
Bit 6	1	Double + Add	$2(P) + P = 3P$
Bit 5	1	Double + Add	$2(3P) + P = 7P$
Bit 4	0	Double	$2(7P) = 14P$
Bit 3	1	Double + Add	$2(14P) + P = 29P$
Bit 2	0	Double	$2(29P) = 58P$
Bit 1	0	Double	$2(58P) = 116P$
Bit 0	1	Double + Add	$2(116P) + P = 233P$

The total computation time is determined by the formula:

$$\text{Total Time} = (\text{Number of Doubles} \times T_D) + (\text{Number of Additions} \times T_A)$$

Substituting the values obtained from the trace:

$$\begin{aligned} \text{Total Time} &= (7 \times 12 \mu s) + (4 \times 18 \mu s) \\ &= 84 \mu s + 72 \mu s \\ &= \mathbf{156 \mu s} \end{aligned}$$

2. Explain why Ethereum PoS is considered Economically Final whereas PoW is only Probabilistically Final? [3 Marks]

Answer: In Proof of Work (PoW), a block is never truly permanent; an attacker with enough hash power can theoretically mine a longer chain and reorg the state, making finality a matter of waiting for enough confirmations (probability).

In Proof of Stake (PoS), once a block is Finalized by a 2/3 supermajority, it is mathematically locked by the protocol's slashing rules. To revert a finalized block, an attacker must cause a Safety Violation by convincing 1/3 of the total staked ETH to double-vote. This results in the automatic burning (slashing) of billions of dollars worth of ETH. Thus, finality is guaranteed by the physical destruction of the attacker's capital, creating an economic barrier that PoW lacks.

3. A miner prioritizes transactions based on the Fee Rate (Satoshis per byte). Consider a Legacy P2PKH transaction that has **2 inputs** and **2 outputs**. In Legacy Bitcoin, a typical input is **148 bytes** and a typical output is **34 bytes**, with an additional **10 bytes** for fixed header data. [3 Marks]

- Calculate the total size of this transaction in **bytes**.
- **Fee Calculation:** If the current market fee rate is **20 satoshis/byte**, calculate the total fee required for this transaction to be confirmed quickly. If $1 \text{ BTC} = 10^8 \text{ satoshis}$, express this fee in **BTC**.

Answer:

- **Transaction Size Calculation:**
 - Size = $(2 \times 148) + (2 \times 34) + 10$
 - Size = $296 + 68 + 10 = \mathbf{374 \text{ bytes}}$.
- **Fee in BTC:**
 - Total Satoshis = $374 \text{ bytes} \times 20 \text{ sats/byte} = 7,480 \text{ satoshis}$.
 - Fee in BTC = $7,480 / 100,000,000 = \mathbf{0.00007480 \text{ BTC}}$.

4. Explain a scenario where the Head of the Chain (determined by LMD GHOST) might be different from the Last Finalized Block (determined by Casper FFG). [3 Marks]

Answer:

- **LMD GHOST (Latest Message Driven Greediest Heaviest Observed Subtree):** This is the Fork Choice Rule. Its role is to help a node decide which branch is the "canonical" head of the chain in the short term. It provides Liveness.
- **Casper FFG (Friendly Finality Gadget):** This is the Finality Layer. Its role is to take the blocks produced by the fork choice and "lock" them permanently after a supermajority (2/3) of stake has attested to them. It provides Safety.

Casper FFG only finalizes blocks at Epoch boundaries (every 32 slots or ~ 6.4 minutes), there is always a gap between the Current Head and the Last Finalized Block.

- **The Head:** As of this second, a block proposer has just added a new block to the tip of the chain. LMD GHOST sees this as the Heaviest tip because of recent attestations.
- **The Finalized Block:** That new head is not yet finalized. It must wait until the end of the current epoch, and then the following epoch, to achieve the 2/3 supermajority required for finalization.
- **The Difference:** If a network partition occurs, LMD GHOST might keep growing a "dirty" chain that never reaches the 2/3 threshold. In this case, the Head continues to move, but the Finalized Block stays stuck in the past until the partition heals.

Part - C

1. A user submits a transaction on three blockchain platforms: Bitcoin, Ethereum, and Hyperledger Fabric. Present a comparison that highlights the following aspects for each platform: [6 Marks]

- Transaction execution flow
- Validation mechanism
- Ordering/consensus process

Answer:

(a) **Bitcoin:**

Execution Flow

- Transaction is broadcast to the network
- Nodes verify inputs using the UTXO model
- Transaction is stored in the mempool
- Miners include it in a block

Validation Mechanism

- Script execution (locking and unlocking scripts)
- Digital signature verification
- Double-spending prevention using UTXO set

Ordering / Consensus

- Transactions are ordered into blocks by miners
- Consensus achieved using **Proof-of-Work (PoW)**
- Longest chain (highest cumulative work) is accepted

(b) **Ethereum:**

Execution Flow

- Transaction is broadcast to the network
- Execution Layer executes transaction using EVM
- State is updated (balances, storage)

- Block proposer includes transaction in a block

Validation Mechanism

- Re-execution of transactions by nodes
- Gas validation and state transition checks

Ordering / Consensus

- Block proposer selected using PoS
- Validators attest to blocks
- Finality achieved using **Gasper (LMD-GHOST + Casper FFG)**

(c) Hyperledger Fabric:

Execution Flow

- Client sends transaction proposal to endorsing peers
- Peers simulate execution (chaincode)
- Read-set and write-set are generated

Validation Mechanism

- Endorsement policy verification
- Read-write conflict check (MVCC)

Ordering / Consensus

- Ordering service orders transactions into blocks
- Uses permissioned consensus (e.g., Raft)
- Peers validate and commit blocks

(d) Comparison Summary

Platform	Execution Timing	Validation	Consensus
Bitcoin	During validation	Script + UTXO	PoW
Ethereum	Before consensus	EVM re-execution	PoS
Fabric	Before ordering	Endorsement + MVCC	Ordering Service

(e) Final Insight:

Bitcoin: Validate → Include → Confirm

Ethereum: Execute → Agree → Finalize

Fabric: Simulate → Order → Validate → Commit

2. A security engineer identifies a vulnerability in a custom-built Merkle Tree implementation where the internal node hashing function is commutative. Specifically, for any two child nodes A and B , the parent node N is computed as $N = H(A \oplus B)$ or $N = H(\text{sort}(A, B))$, effectively ensuring that $H(A||B) = H(B||A)$. [6 Marks]

(a) Explain the technical flaw regarding Topological Malleability.

Answer: In a secure Merkle Tree, the path from a leaf to the root is a fixed sequence. Because standard concatenation is non-commutative ($A + B \neq B + A$), the Merkle Root acts as a commitment to both the content of the transactions and their specific order.

When a hash is commutative, the Root only commits to the set of data, not the sequence. This is called Topological Malleability: the attacker can move a leaf to a different position in the tree topology without changing the resulting Root. (Note: The attack is NOT breaking the hash function and verification passes but semantic meaning changes)

(b) Demonstrate a scenario where an attacker, provided with a valid Merkle Root and an inclusion proof for an honest transaction T_{honest} , can generate a mathematically valid proof for a "phantom" transaction T_{phantom} at a different leaf index.

Suppose we have a simple Merkle Tree with two transactions: T_1 (at Index 0) and T_2 (at Index 1).

- Honest Root Calculation: $Root = H(T_1 || T_2)$
- The Flaw: $H(T_1 || T_2) = H(T_2 || T_1)$

An attacker wants to prove that T_2 actually occurred at Index 0.

- The Verifier asks for a proof for Index 0.
- The attacker provides the data for T_2 and the sibling T_1 .
- The Verifier calculates: $Calculated_Root = H(T_2 || T_1)$.
- Because the hash is commutative, $H(T_2 || T_1)$ matches the stored $Root$.
- The Verifier is mathematically convinced that T_2 is located at Index 0, effectively overwriting or duplicating the history of the ledger.

- (c) Discuss why this design violates the fundamental binding property required for non-repudiation in a distributed ledger.

Non-repudiation relies on the fact that once a piece of data is anchored to a Merkle Root at a specific index, it cannot be moved or denied. By removing the requirement for left/right ordering, the developer has allowed Position-Agnostic Inclusion Proofs.

In a real-world system (like Ethereum), this would allow an attacker to take a Withdrawal transaction from a legitimate user and re-prove it at a different index to potentially double-spend or claim rewards that were already distributed.

3. In the PBFT system ($N = 3f + 1$), how is the View Change algorithm triggered when a Primary or Leader fails? Demonstrate mathematically, using quorum intersection logic, how this mechanism guarantees Liveness despite the presence of f Byzantine nodes. [6 Marks]

Answer: In PBFT, the View Change algorithm is triggered when replicas suspect that the current Primary is faulty, usually because it fails to send a valid PRE-PREPARE message or the protocol makes no progress within a timeout period. When this happens, a view-change message simulcast by replica i to move to view $v + 1$ is given by:

$$\langle \text{VIEW-CHANGE}, v + 1, n, C, P, i \rangle_{\sigma_i}$$

Where,

- $v + 1$: New view number
- n : Sequence number of the latest stable checkpoint
- i : Replica identifier
- σ_i : Digital signature (or authenticator) of replica i
- C : Set of checkpoint proofs for checkpoint n
- P : a set of information about requests that are prepared but not yet committed, typically tuples like (n, d, v) , where n : sequence number, d : digest of the request, and v : view in which it was prepared

A new view is started when the new Primary collects at least:

$$2f + 1$$

valid VIEW-CHANGE messages.

PBFT guarantees liveness: In PBFT, the total number of replicas is:

$$N = 3f + 1$$

and at most f replicas can be Byzantine. So, the number of honest replicas is at least:

$$N - f = (3f + 1) - f = 2f + 1$$

Thus, even if all f Byzantine nodes refuse to cooperate, the honest replicas alone are sufficient to form the required view-change quorum:

$$2f + 1$$

Hence, the protocol cannot remain stuck forever with a faulty Primary.

Quorum intersection argument: PBFT uses quorums of size:

$$2f + 1$$

Now consider two such quorums, Q_1 and Q_2 . Their minimum intersection is:

$$|Q_1 \cap Q_2| \geq |Q_1| + |Q_2| - N$$

Substituting:

$$\begin{aligned} |Q_1 \cap Q_2| &\geq (2f + 1) + (2f + 1) - (3f + 1) \\ &= 4f + 2 - 3f - 1 \\ &= f + 1 \end{aligned}$$

So any two quorums intersect in at least:

$$f + 1$$

replicas.

Since at most f replicas are Byzantine, at least one replica in that intersection must be honest.

Part - D

1. In an Ethereum transaction, the source address is not explicitly stored in the transaction data to save space. Instead, the network recovers the public key from the signature. Consider the elliptic curve over the finite field $\mathbb{F}_{17}$: [10 Marks]

$$y^2 \equiv x^3 + 2x + 2 \pmod{17}$$

Let the generator point be:

$$G = (5, 1)$$

Suppose for a signed transaction:

- transaction hash: $z = 2$
- signature components: $(r, s) = (10, 3)$
- ephemeral point: $R = k \cdot G = (7, 11)$

Find the public key Q using the public key recovery formula:

$$Q = r^{-1}(sR - zG) \pmod{17}$$

Answer:

- (a) To recover the public key Q , we must isolate d and then convert it to a point $Q = d \cdot G$.

- Start with the signature equation:

$$s \equiv k^{-1}(z + rd) \pmod{n}$$

- Multiply both sides by the nonce k :

$$sk \equiv z + rd \pmod{n}$$

- Isolate the term containing the private key d :

$$rd \equiv sk - z \pmod{n}$$

- Multiply by the modular inverse of r (r^{-1}):

$$d \equiv r^{-1}(sk - z) \pmod{n}$$

- To find the public key Q , multiply the entire scalar equation by the generator point G :

$$dG = r^{-1}(s(kG) - zG) \pmod{n}$$

- Substitute $Q = dG$ and $R = kG$:

$$Q = r^{-1}(sR - zG) \pmod{n}$$

(b) To calculate the public key Q :

- Compute r^{-1} Since:

$$r = 1 \Rightarrow r^{-1} = 1$$

- Compute $sR = 3R$ Given:

$$R = (7, 11)$$

First compute $2R$ using point doubling:

$$\lambda = \frac{3x_1^2 + a}{2y_1} \pmod{17}$$

$$\lambda = \frac{3(7^2) + 2}{2(11)} = \frac{149}{22} \pmod{17}$$

$$149 \equiv 13, \quad 22 \equiv 5 \pmod{17}$$

$$5^{-1} \equiv 7 \pmod{17}$$

$$\lambda = 13 \cdot 7 = 91 \equiv 6 \pmod{17}$$

$$x_3 = \lambda^2 - 2x_1 = 6^2 - 14 = 36 - 14 = 22 \equiv 5$$

$$y_3 = \lambda(x_1 - x_3) - y_1 = 6(7 - 5) - 11 = 12 - 11 = 1$$

$$2R = (5, 1)$$

Now compute $3R = 2R + R$:

$$(5, 1) + (7, 11)$$

$$\lambda = \frac{11 - 1}{7 - 5} = \frac{10}{2} \pmod{17}$$

$$2^{-1} \equiv 9 \pmod{17}$$

$$\lambda = 10 \cdot 9 = 90 \equiv 5 \pmod{17}$$

$$x_3 = 5^2 - 5 - 7 = 25 - 12 = 13$$

$$y_3 = 5(5 - 13) - 1 = 5(-8) - 1 = -41 \equiv 10$$

$$sR = 3R = (13, 10)$$

- Compute $zG = 2G$

$$G = (5, 1)$$

$$\lambda = \frac{3(5^2) + 2}{2(1)} = \frac{77}{2}$$

$$77 \equiv 9, \quad 2^{-1} \equiv 9 \pmod{17}$$

$$\lambda = 9 \cdot 9 = 81 \equiv 13$$

$$x_3 = 13^2 - 10 = 169 - 10 = 159 \equiv 6$$

$$y_3 = 13(5 - 6) - 1 = -14 \equiv 3$$

$$zG = 2G = (6, 3)$$

- Compute $sR - zG$

$$(13, 10) - (6, 3) = (13, 10) + (6, -3)$$

$$-3 \equiv 14 \pmod{17}$$

$$(13, 10) + (6, 14)$$

$$\lambda = \frac{14 - 10}{6 - 13} = \frac{4}{-7}$$

$$-7 \equiv 10, \quad 10^{-1} \equiv 12$$

$$\lambda = 4 \cdot 12 = 48 \equiv 14$$

$$x_3 = 14^2 - 13 - 6 = 196 - 19 = 177 \equiv 7$$

$$y_3 = 14(13 - 7) - 10 = 84 - 10 = 74 \equiv 6$$

$$sR - zG = (7, 6)$$

- Compute Q :

$$Q = r^{-1}(sR - zG) = 1 \cdot (7, 6)$$

$$Q = (7, 6)$$

2. Explain the end-to-end Ethereum smart contract life cycle / execution flow in detail, starting from writing a Solidity contract and ending with a permanent state entity. Use pictures, **flowchart**, and formal notations where applicable, including the contract address derivation and the block header root structures. [10 Marks]

Your analysis must strictly detail:

- Artifact Generation:** The transition from Solidity to machine-level primitives.
- Deployment Mechanics:** The cryptographic derivation of addresses and the role of Init Code.
- Network Propagation & Validation:** How the Mempool and Proposer layers maintain protocol rules.
- EVM State Transitions:** The mathematical commitment of state changes via Merkle Patricia Tries.
- Post-Deployment Interaction:** The encoding mechanics of `set()` and `get()` functions.
- Transaction Receipt & Finalization**

Answer:

- Development and Compilation Artifacts:** The lifecycle begins in a high-level language (Solidity). Since the EVM is a stack-based machine, the compiler (solc) must decompose logic into two distinct artifacts:
 - **Bytecode ($\mathcal{B}$):** A sequence of hexadecimal opcodes (e.g., 0x60806040...). This is split into Creation Bytecode (executed once during deployment) and Runtime Bytecode (stored on-chain).
 - **Application Binary Interface (ABI):** A JSON-encoded schema that maps human-readable function signatures to 4-byte Function Selectors (σ) and defines data encoding rules.
- Deployment Transaction & Address Derivation:** A user initiates deployment by broadcasting a transaction T where the recipient address (to) is null (0x0).
 - **Data Payload:** The data field contains the Initialization Code (Constructor logic).
 - **Address Derivation:** The contract address α is not random; it is a deterministic result of the sender's address A and their account nonce n :

$$\alpha = B_{96..255}(\text{Keccak256}(\text{RLP}(A, n)))$$

(Where $B_{96..255}$ denotes the last 160 bits/20 bytes of the hash).

(c) **Network Validation & The Mempool:** Upon broadcast, nodes perform "pre-execution" checks to protect the network from DoS attacks:

- **Signature Verification:** Recovering the public key Q from (r, s, v) to ensure A authorized the deployment.
- **Nonce Check:** Ensuring $T_{nonce} = \text{Account}_{nonce}$.
- **Intrinsic Gas Check:** Ensuring $\text{Gas}_{limit} \geq 21,000 + \text{data_cost}$.
- **Balance Check:** Ensuring $\text{Balance}(A) \geq (\text{Gas}_{limit} \times \text{Gas}_{price}) + \text{Value}$.

If valid, T enters the Mempool. A Block Proposer eventually selects T , packaging it into an Execution Payload.

(d) **EVM Execution & State Commitment:** The EVM executes the Init Code. This code sets up the initial state and, at the end of the execution, uses the RETURN opcode to pass the Runtime Bytecode back to the EVM to be stored at address α .

Ethereum represents its global state as a Merkle Patricia Trie (MPT). The lifecycle is finalized when the new block header is produced, containing three critical roots:

- **stateRoot:** A commitment to the entire world state (Account balances, code hashes, storage).
- **transactionsRoot:** A commitment to all transactions in the block.
- **receiptsRoot:** A commitment to the outcomes (logs, gas used, success/failure status).

(e) **Post-Deployment Interaction (set vs get):** Once live, users interact with the contract using the ABI to encode their intent into the calldata.

- **State-Changing (set):** The user sends a transaction. The first 4 bytes of calldata is the selector σ :

$$\sigma = \text{Keccak256}(\text{"set(uint256)"}_{0..3})$$

The EVM executes the runtime bytecode, moves the Program Counter (PC) to the identified function, and updates the Storage Trie ($\sigma_{storage}$). This requires gas and consensus finality.

- **Read-Only (get):** The user performs an `eth_call`. Since this does not modify the state, it is executed locally by the node's EVM. It returns the value from the contract's storage without a transaction fee or block inclusion.

(f) **Transaction Receipt & Finalization:** After execution, a Transaction Receipt R is generated:

$$R = \{\text{status, cumulativeGasUsed, logs, bloomFilter}\}$$

The contract is considered "live" immediately upon block inclusion, but reaches Economic Finality once the Consensus Layer (Gasper) checkpoints the epoch (typically 12.8 minutes post-inclusion).

Table 2: End-to-End Ethereum Smart Contract Lifecycle Summary

Stage	Primary Operator	Key Primitive
Compilation	Developer / solc	Bytecode & ABI
Address Creation	EVM / RLP	$\alpha = \text{hash}(A, n)$
Validation	P2P Nodes	Signature & Nonce
Execution	Block Proposer / EVM	State Transition ($\sigma \rightarrow \sigma'$)
Finalization	Consensus Layer	Casper FFG / LMD GHOST

3. Write the research paper that you have studied for your blog writing and presentation. Based on your understanding, **briefly** answer the following: [10 Marks]

(a) Clearly describe the problem statement addressed in the paper. Your answer must include the Title of the paper, Conference name, and Year of publication.

- (b) Explain the solution or methodology proposed by the authors to address the problem.
- (c) Present your individual understanding and key findings after reading the paper. Highlight what you personally learned (not the group's findings).
- (d) Propose your own solution approach or improvement for the problem discussed in the paper. Clearly explain how your approach differs from or enhances the original solution.